ls

- ls - list the contents of present working directory
- ls path - list the contents of given path
- ls -l - list the contents in detail format
    - type and permissions
    - Types of files
    - Permissions of files
        - r - read, w - write, x - execute
        - (rwx)user/owner, (rwx)group, (rwx)others
    - link count
    - user/owner
    - group
    - size
    - timestamp
    - name
- ls -a - list all contents along with hidden
- ls -A - list all contents along with hidden except . and ..
- ls -i - list contents with indoe number
    - inode number is unique number given to every file
- ls -s - list content with size (number of blocks)
- ls -S - list content in descending order of their sizes

File Permissions/Mode

- File permission types: r (read), w (write), x (execute)
- Permission levels: u (user), g (group), o (other)
- File mode: u-rwx g-rwx o-rwx
- Example: hello.txt -- user can read/write and execute, group can read and execute, other can only read.
    - hello.txt mode: u=rwx g=r-x o=r--
        - rwx r-x r--
        - 111 101 100 = 754
    - chmod 754 hello.txt

chmod

- chmod permissions filename
- Permissions:
    - +x/r/w - add permission to all
    - -x/r/w - remove permission of all
    - u+x/r/w - add permission to user
    - u-x/r/w - remove permission of user
    - g+x/r/w - add permission to group
    - g-x/r/w - remove permission of group
    - o+x/r/w - add permission to others
    - o-x/r/w - remove permission of others
    - 0774 - apply rwx to user, rwx to group and r to others

chown

- sudo chown username filename
- sudo chown :groupname filename
- sudo chown username:groupname filename

Directory permissions

- r (read): Can read directory entries.
  - ls command can list the directory.
  - readdir() can get directory entry.
- w (write): Can add new directory entry, modify existing directory entry, or delete directory entry.
  - new directory entry: when create new sub-directory (e.g. mkdir) or file (e.g. touch, ...).
  - modify existing directory entry: rename file or sub-directory.
  - delete directory entry: delete file or sub-directory (e.g. rm, rmdir, ...)
- x (execute): Can browse the directory
  - cd command (chdir() syscall) will work

Regular Expressions

- Find a pattern in text file(s).
- Regular expressions are patterns used to match character combinations in strings.
- A regular expression pattern is composed of simple characters, or a combination of simple and special characters e.g. /abc/, /ab*c/
- Pattern is given using regex wild-card characters.
  - Basic wild-card characters
    - $ - find at the end of line.
    - ^ - find at the start of line.
    - [ ] - any single char in give range or set of chars
    - [^] - any single char not in give range or set of chars
    - . - any single character
    - zero or more occurrences of previous character
  - Extended wild-card characters
    - ? - zero or one occurrence of previous character
    - one or more occurrences of previous character
    - {n} - n occurrences of previous character
    - {,n} - max n occurrences of previous character
    - {m,} - min m occurrences of previous character
    - {m,n} - min m and max n occurrences of previous character
    - () - grouping (chars)
    - (|) - find one of the group of characters

grep

- Regex commands
  - grep - GNU Regular Expression Parser - Basic wild-card
  - egrep - Extended Grep - Basic + Extended wild-card
  - fgrep - Fixed Grep - No wild-card

- Command syntax
  - grep "pattern" filepath
  - grep [options] "pattern" filepath
    - -c : count number of occurrences
    - -v : invert the find output
    - -i : case insensitive search
    - -w : search whole words only
    - -R : search recursively in a directory
    - -n : show line number.

sed command

syntax

```
sed [OPTIONS] 'COMMAND' [INPUTFILE...]
```

OPTIONS

- -i : Edit the file in-place (overwrite)
- -n : Suppress automatic printing of lines.
- -e : Allows multiple commands.
- -f : Reads sed commands from a file.
- -r : Use extended regular expressions.

COMMAND

- s/old_word/new_word/ - substitute first occurrence of old_word by new_word
- s/old_word/new_word/n - substitute nth occurrence of old_word by new_word
- s/old_word/new_word/g - substitute all occurrences of old_word by new_word
- s/old_word/new_word/gi - case sensitive substitute all occurrences of old_word by new_word
- s/old_word/new_word/ng - substitute from nth occurrence of old_word by new_word
- n s/old_word/new_word/ - substitute first occurrence of old_word by new_word on nth line
- m,n s/old_word/new_word/ - substitute first occurrence of old_word by new_word from mth to nth line
- m,$ s/old_word/new_word/ - substitute first occurrence of old_word by new_word from mth to last line
- s/old_word/new_word/p - duplicate replaced lines
- nd - delete nth line
- $d - delete last line
- m,nd - delete mth to nth line
- m,$d - delete mth to last line
- /pattern/d - delete pattern matching line
- = - print file with line numbers

```
cat > sample.txt
Linux is a powerful operating system.
Unix is the origin of Linux and many Unix systems are still in use.
Linux and Unix are both multiuser systems.
```

```
This system is used in servers, and this system is very reliable.
Many developers prefer Linux because Linux is open source.
Unix systems were developed earlier than Linux systems.
This line contains Linux Linux Linux multiple times.
Unix Unix systems are known for stability.
The system administrator manages the Linux system.
EOF
```

```
# replace words start with L
sed 's/\bL[a-zA-Z]*/OS/g' sample.txt

# Replace only whole word "Linux"
sed 's/\bLinux\b/OS/g' sample.txt

# Replace lines starting with "Unix"
sed 's/^Unix/OS/' sample.txt

# Replace lines ending with "system."
sed 's/system\.$/OS./' sample.txt

# Replace multiple spaces with one space
sed 's/  */ /g' sample.txt

# Print lines with either Linux OR Unix
sed -n '/Linux\|Unix/p' sample.txt

# Replace digits
sed 's/[0-9]/X/g' sample.txt

# Insert text before line 3
sed '3i\new text' filename

# Insert text after line 3
sed '3a\new text' filename
```


Regex symbols

- ^ - Start of line
- $ - End of line
- . - Any character
- *- 0 or more
- + - 1 or more
- ? - 0 or 1
- [] - Character class
- \b - Word boundary
- | - OR
- () - Grouping

VI Editor

- sudo apt-get install vim
- VI editor works in two modes
    - command mode
    - insert mode
- press i - to go into insert mode
- press Esc - to go into command node
- VI editor commands:
    - :w - write/save into file
    - :q - quit vi editor
    - :wq - save and quit
    - :y - to copy current line
    - :ny - to copy nth line
    - yy - to copy current line
    - nyy - copy n lines from current line
    - :m,ny - copy fomr mth line to nth line
    - :d - to cut current line
    - :nd - to cut nth line
    - dd - to cut current line
    - ndd - cut n lines from current line
    - :m,nd - cut fomr mth line to nth line
    - press p - to paste copied line on next line of current line
    - yw - copy from current position upto next word
    - yiw - copy current word
    - y$ - copy from cursor position upto end of line
    - y^ - copy from cursor position upto start of line
    - vim -o - to open multiple files horizontally
    - vim -O - to open multiple files vertically
    - ctrl + ww - to go into next file
    - /pattern - to search the pattern

    - n - to go on next occurance
    - 😒/pattern1/pattern2/ - find and replace only first occurance of current line
    - 😒/pattern1/pattern2/g - find and replace all occurances of current line
    - :%s/pattern1/pattern2/g - find and replace all occurances of file
- To do setting in vi editor
    - create file into home directory as below:

```
vim ~/.vimrc
```

- add below content into it

```
set number
set autoindent
set tabstop=4
set nowrap
```